

Divisão e conquista

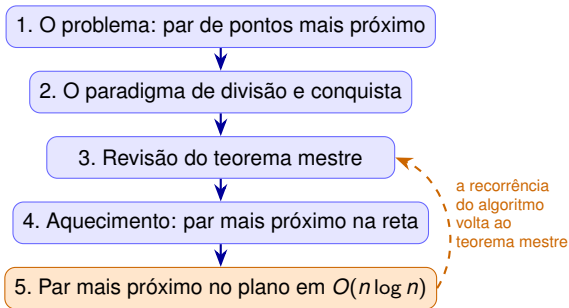
Teorema mestre e o par de pontos mais próximo

Prof. Dr. André Yoshiaki Kashiwabara

Departamento de Computação — Universidade Federal de Sergipe

29 de setembro de 2026

O que você vai aprender hoje



O problema motivador

Problema:

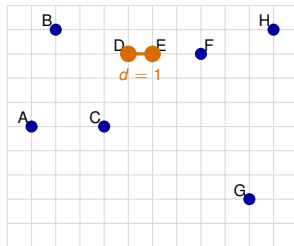
Dados n pontos no plano, encontrar os dois que estão mais próximos um do outro.

- controle de tráfego aéreo: dois aviões perto demais;
- simulação física: detecção de colisões;
- agrupamento: primeiro passo da ligação simples.

Solução (a desta aula):

dividir o plano ao meio, resolver cada metade e examinar só uma *faixa estreita* perto da divisa.

Para aprofundar: Levitin (2012), §3.3 e §5.5 [19]; Kleinberg e Tardos (2006), §5.4 [16].



exemplo condutor: 8 pontos, par mais próximo $\{D, E\}$

Definição

Dado $P = \{p_1, \dots, p_n\}$, com $p_i = (x_i, y_i)$ e $n \geq 2$, encontrar $i \neq j$ que minimizam a distância euclidiana

$$d(p_i, p_j) = \sqrt{(x_i - x_j)^2 + (y_i - y_j)^2}.$$

Intuição

Em uma dimensão basta ordenar e olhar vizinhos consecutivos. No plano não existe uma ordem que coloque lado a lado os pontos próximos: ordenar por x deixa B ao lado de C , que estão a $\sqrt{2}0$ um do outro.

Para aprofundar: Cormen et al. (2009), §33.4 [5]; Shamos e Hoey (1975), que propuseram o problema e o algoritmo [25].

```
double forcaBruta( Ponto P[], int n) {  
    double d = INFINITY;  
    for (int i = 0; i < n; ++i)  
        for (int j = i+1; j < n; ++j) {  
            double dij = dist( P[i], P[j]);  
            if (dij < d) d = dij;  
        }  
    return d;  
}
```

Número de distâncias calculadas:

$$\sum_{i=0}^{n-1} (n-1-i) = \binom{n}{2} = \frac{n(n-1)}{2} \in \Theta(n^2).$$

É a estratégia da Unidade 1: correta, simples e quadrática.

No exemplo condutor: $\binom{8}{2} = 28$ distâncias.

Para aprofundar: Levitin (2012), §3.3, *Closest-Pair Problem by Brute Force* [19].

Quanto custa $\Theta(n^2)$ na prática?

n	pares $\binom{n}{2}$	tempo a 10^9 pares/s	$n \log_2 n$ a 10^9 /s
10^3	5×10^5	0,5 ms	0,01 ms
10^5	5×10^9	5 s	1,7 ms
10^6	5×10^{11}	≈ 8 min	20 ms
10^7	5×10^{13}	≈ 14 h	0,23 s

Pergunta da aula

Existe um algoritmo que *não* examine todos os pares? A resposta é sim, e ele custa $O(n \log n)$. Para chegar lá precisamos de uma técnica de projeto (divisão e conquista) e de uma ferramenta de análise (o teorema mestre).

Para aprofundar: Estimativas de ordem de grandeza; medições reais no tutorial da aula. Kleinberg e Tardos (2006), §5.4 [16].

O que aprendemos

- O par mais próximo tem aplicações em geometria, simulação e agrupamento.
- A força bruta examina $\binom{n}{2}$ pares: $\Theta(n^2)$.
- Para $n = 10^6$, isso já leva minutos; queremos $O(n \log n)$.
- Ordenar por x *sozinho* não resolve no plano.

Para aprofundar: Levitin (2012), §3.3 [19].

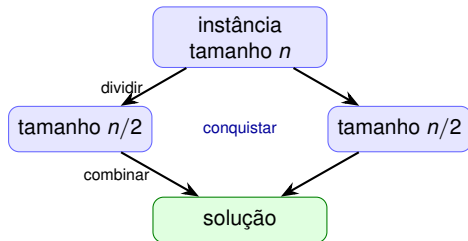
Divisão e conquista

Problema:

Algoritmos incrementais (inserção, força bruta) avançam um elemento por vez e tendem a custar $\Theta(n^2)$.

Solução:

Quebrar a instância em pedaços de tamanho n/b , resolvê-los recursivamente e *combinar* as respostas. Se dividir e combinar forem baratos, o ganho se multiplica a cada nível da recursão.



Para aprofundar: Feofiloff, aula *Divisão e conquista* [6]; Levitin (2012), cap. 5 [19].

Definição

Um algoritmo de divisão e conquista resolve uma instância em três fases:

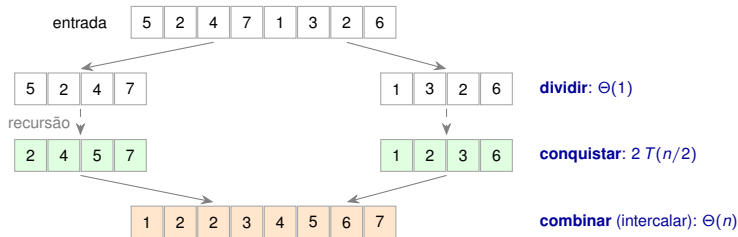
1. **dividir** a instância em a instâncias menores do mesmo problema;
2. **conquistar**: resolver cada uma recursivamente (ou diretamente, se pequena);
3. **combinar** as soluções parciais em uma solução da instância original.

Intuição

A recursão é a parte fácil, porque “resolve sozinha”. Todo o engenho fica em *dividir* e *combinar*, e é daí que vem o custo $f(n)$ que aparece na recorrência.

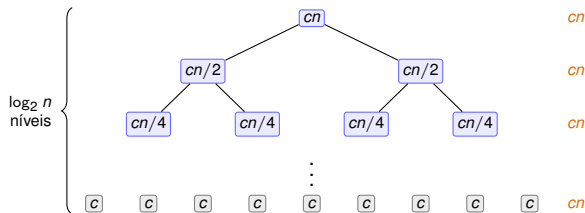
Para aprofundar: Cormen et al. (2009), §2.3.1 [5]; Feofiloff, aula *Divisão e conquista* [6].

Exemplo-âncora: mergesort



$$T(n) = 2 T(n/2) + \Theta(n), \quad T(1) = \Theta(1).$$

Para aprofundar: Cormen et al. (2009), §2.3, Fig. 2.4 [5]; Levitin (2012), §5.1 [19].



- No nível j há 2^j nós, cada um com custo $cn/2^j$.
 - Soma por nível: $2^j \cdot cn/2^j = cn$.
 - São $\log_2 n + 1$ níveis.
- $T(n) = cn(\log_2 n + 1) \in \Theta(n \log n)$.

Desenhar a árvore para cada algoritmo cansa, e o teorema mestre faz essa soma por nós.

Para aprofundar: Cormen et al. (2009), §2.3.2, Fig. 2.5, e §4.4 [5].

O que aprendemos

- Três fases: dividir, conquistar (recursão) e combinar.
- O custo não recursivo de dividir e combinar forma o termo $f(n)$.
- a subproblemas de tamanho n/b dão $T(n) = a T(n/b) + f(n)$.
- A árvore de recursão soma o custo nível a nível; no mergesort, cn por nível em $\log n$ níveis.

Para aprofundar: Levitin (2012), cap. 5 [19]; Cormen et al. (2009), §4.4 [5].

Revisão do teorema mestre

Problema:

Toda análise de divisão e conquista termina em uma recorrência

$$T(n) = a T(n/b) + f(n),$$

e resolvê-la por substituição ou árvore de recursão é trabalhoso.

Solução:

Um “livro de receitas”: comparar $f(n)$ com $n^{\log_b a}$, que é o número de folhas da árvore, e ler a resposta em um de três casos.

$a \geq 1$	número de subproblemas
$b > 1$	fator de redução do tamanho
$f(n)$	custo de dividir e combinar

Para aprofundar: Cormen et al. (2009), §4.5 [5]; Feofiloff, aula *Recorrências*, seção *Teorema Mestre* [6].

Teorema (Levitin, 2012)

Se $T(n) = aT(n/b) + f(n)$ com $a \geq 1$, $b \geq 2$ e $f(n) \in \Theta(n^d)$, $d \geq 0$, então

$$T(n) \in \begin{cases} \Theta(n^d) & \text{se } a < b^d \quad (\text{a raiz domina}), \\ \Theta(n^d \log n) & \text{se } a = b^d \quad (\text{todos os níveis empatam}), \\ \Theta(n^{\log_b a}) & \text{se } a > b^d \quad (\text{as folhas dominam}). \end{cases}$$

Intuição

O nível j da árvore custa $a^j (n/b^j)^d = n^d (a/b^d)^j$. Temos uma série geométrica de razão $r = a/b^d$: se $r < 1$, ela decresce e a soma é dominada pelo primeiro termo; se $r = 1$, são $\log_b n$ termos iguais; se $r > 1$, ela cresce e o último termo (as folhas) domina.

Para aprofundar: Levitin (2012), cap. 5, *Master Theorem* [19]; Kleinberg e Tardos (2006), §5.2 [16].

Os três casos vistos na árvore



$a < b^d$
raiz domina
 $\Theta(f(n))$



$a = b^d$
níveis iguais
 $\Theta(f(n) \log n)$



$a > b^d$
folhas dominam
 $\Theta(n^{\log_b a})$

Cada barra é a soma dos custos de um nível. A resposta é a da parte mais pesada da árvore.

Para aprofundar: Cormen et al. (2009), §4.6, Fig. 4.7 [5].

Teorema 4.1 (Cormen et al., 2009)

Sejam $a \geq 1$, $b > 1$ e $T(n) = aT(n/b) + f(n)$. Compare $f(n)$ com $n^{\log_b a}$:

1. se $f(n) = O(n^{\log_b a - \varepsilon})$ para algum $\varepsilon > 0$, então $T(n) = \Theta(n^{\log_b a})$;
2. se $f(n) = \Theta(n^{\log_b a})$, então $T(n) = \Theta(n^{\log_b a} \lg n)$;
3. se $f(n) = \Omega(n^{\log_b a + \varepsilon})$ para algum $\varepsilon > 0$ e $a f(n/b) \leq c f(n)$ para alguma constante $c < 1$ e todo n grande (regularidade), então $T(n) = \Theta(f(n))$.

O que muda em relação à versão com n^d

f não precisa ser um polinômio, mas precisa ser *polinomialmente* menor ou maior que $n^{\log_b a}$ (o fator n^ε). Pisos e tetos ($\lfloor n/2 \rfloor$, $\lceil n/2 \rceil$) não alteram a resposta.

Para aprofundar: Cormen et al. (2009), §4.5, Teorema 4.1, e §4.6.2 [5].

algoritmo	recorrência	a	b	d	resultado
busca binária	$T(n/2) + \Theta(1)$	1	2	0	$a = b^d: \Theta(\log n)$
mergesort	$2T(n/2) + \Theta(n)$	2	2	1	$a = b^d: \Theta(n \log n)$
máximo por D&C	$2T(n/2) + \Theta(1)$	2	2	0	$a > b^d: \Theta(n)$
multiplicação ingênua	$4T(n/2) + \Theta(n)$	4	2	1	$a > b^d: \Theta(n^2)$
Karatsuba	$3T(n/2) + \Theta(n)$	3	2	1	$a > b^d: \Theta(n^{\log_2 3}) \approx n^{1,585}$
Strassen	$7T(n/2) + \Theta(n^2)$	7	2	2	$a > b^d: \Theta(n^{\log_2 7}) \approx n^{2,807}$

Lição para as próximas aulas

Quando as folhas dominam, cortar *um* subproblema ($a: 4 \rightarrow 3, 8 \rightarrow 7$) muda o expoente. Karatsuba e Strassen fazem exatamente isso.

Para aprofundar: Levitin (2012), §5.4 [19]; Cormen et al. (2009), §4.2 (Strassen) [5]; Kleinberg e Tardos (2006), §5.5 (Karatsuba) [16].

recorrência	por que falha	o que fazer
$T(n) = T(n-1) + n$	não divide por b	somar direto: $\Theta(n^2)$
$T(n) = T(n/3) + T(2n/3) + n$	subproblemas desiguais	árvore de recursão: $\Theta(n \log n)$
$T(n) = n T(n/2) + 1$	a não é constante	substituição / árvore
$T(n) = 2T(n/2) + n \lg n$	$n \lg n$ não é $n^{1+\varepsilon}$	extensão do caso 2

Extensão do caso 2

Se $f(n) = \Theta(n^{\log_b a} \lg^k n)$ com $k \geq 0$, então $T(n) = \Theta(n^{\log_b a} \lg^{k+1} n)$.

Assim, $2T(n/2) + n \lg n = \Theta(n \lg^2 n)$. Vamos reencontrar essa recorrência no par mais próximo.

Para aprofundar: Cormen et al. (2009), §4.5 (exemplo da lacuna) e Exercício 4.6-2 [5].

O que aprendemos

- Compare o custo da raiz $f(n)$ com o das folhas $n^{\log_b a}$.
- Raiz domina: $\Theta(f(n))$. Empate: $\Theta(f(n) \log n)$. Folhas dominam: $\Theta(n^{\log_b a})$.
- Na versão geral a diferença precisa ser polinomial, e o caso 3 pede regularidade.
- Fora da forma $aT(n/b) + f(n)$, volte à árvore de recursão.

Para aprofundar: Cormen et al. (2009), §4.5 [5]; Levitin (2012), cap. 5 [19].

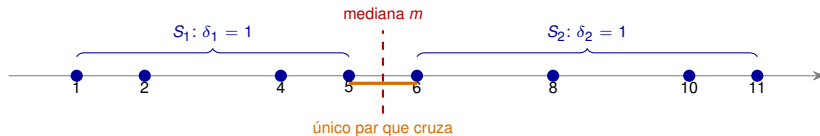
Aquecimento: pontos na reta

Problema:

Dados n números reais $x_1, \dots, x_n$, achar o par mais próximo. Ordenar e comparar vizinhos resolve em $\Theta(n \log n)$, mas essa ideia *não* se generaliza para o plano.

Solução:

Resolver com divisão e conquista e observar *onde* está o trabalho da combinação. É esse o esqueleto que levaremos para o plano.



Para aprofundar: Levitin (2012), §5.5 [19]; Shamos e Hoey (1975), §2 [25].

Algoritmo (entrada já ordenada)

1. **Dividir** na mediana: S_1 (metade esquerda) e S_2 (metade direita).
2. **Conquistar**: δ_1 e δ_2 recursivamente.
3. **Combinar**: $\delta = \min\{\delta_1, \delta_2, \min S_2 - \max S_1\}$.

Por que a combinação é barata

Um par que cruza a divisa e é mais curto que δ_1 e δ_2 só pode ser o do *último* de S_1 com o *primeiro* de S_2 : há um único candidato.

$T(n) = 2T(n/2) + \Theta(1) \Rightarrow$ caso $a > b^d$: $\Theta(n)$, mais $\Theta(n \log n)$ da ordenação inicial.

Para aprofundar: Levitin (2012), §5.5 [19]; Shamos e Hoey (1975), §2 [25].

O que aprendemos

- O par ótimo está inteiro em S_1 , inteiro em S_2 ou *cruza* a divisa.
- Pares que cruzam só importam se forem mais curtos que $\delta = \min\{\delta_1, \delta_2\}$.
- Na reta existe um só candidato que cruza; no plano, quantos existem?

Para aprofundar: Shamos e Hoey (1975), §2 [25].

Par mais próximo no plano

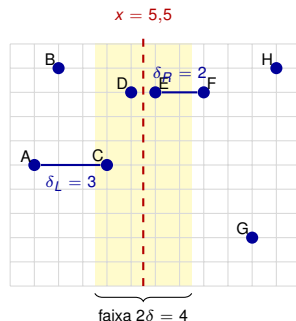
Problema:

Dividindo por uma reta vertical, podem existir até $\frac{n}{2} \cdot \frac{n}{2}$ pares que cruzam a divisa. Examinar todos nos devolve a $\Theta(n^2)$.

Solução:

Com $\delta = \min\{\delta_L, \delta_R\}$, só importam pontos a menos de δ da reta, e cada um deles precisa ser comparado com **no máximo 7** vizinhos.

Para aprofundar: Cormen et al. (2009), §33.4 [5]; Kleinberg e Tardos (2006), §5.4 [16].



Algoritmo (Shamos e Hoey, 1975; versão de Cormen et al.)

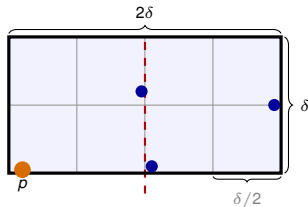
Entrada: X (pontos ordenados por x) e Y (os mesmos, ordenados por y).

1. **Base:** se $n \leq 3$, força bruta.
2. **Dividir:** reta vertical na mediana de X ; separar Y em Y_L e Y_R *mantendo a ordem por y* , em tempo $\Theta(n)$.
3. **Conquistar:** $\delta_L = \text{PMP}(X_L, Y_L)$ e $\delta_R = \text{PMP}(X_R, Y_R)$; $\delta = \min\{\delta_L, \delta_R\}$.
4. **Combinar:** $Y' \leftarrow$ pontos de Y com $|x - x_m| < \delta$ (já ordenados por y); comparar cada ponto de Y' com os **7 seguintes**; atualizar δ .

Intuição

A faixa pode conter todos os n pontos, mas mesmo assim o trabalho é $\leq 7n$ comparações, porque δ limita quantos pontos cabem perto de cada um.

Para aprofundar: Cormen et al. (2009), §33.4 [5]; Shamos e Hoey (1975) [25].



- Seja p um ponto da faixa. Se q vem depois de p em Y' e $d(p, q) < \delta$, então q está no retângulo $2\delta \times \delta$ que começa na altura de p .
- Divida o retângulo em 8 quadrados de lado $\delta/2$. Cada quadrado fica de um só lado da reta.
- Dois pontos no mesmo quadrado distam no máximo $\frac{\delta}{2}\sqrt{2} < \delta$, o que contradiz $\delta_L, \delta_R \geq \delta$.
- Logo há no máximo 8 pontos no retângulo: p e **7 outros**.

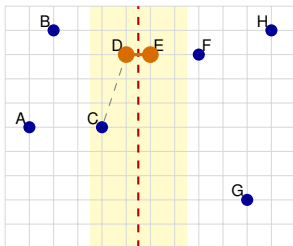
Para aprofundar: Cormen et al. (2009), §33.4, Fig. 33.11 [5]; Kleinberg e Tardos (2006), §5.4, (5.10) [16].

```
/* F[0..f-1]: pontos com |x - divisor.x| < d, ja' em ordem de y */
int f = 0;
for (int i = 0; i < n; ++i)
    if (fabs( Y[i].x - divisor.x) < d) F[f++] = Y[i];

for (int i = 0; i < f; ++i)
    for (int j = i+1; j < f && F[j].y - F[i].y < d; ++j) {      /* no maximo 7 */
        double dij = dist( F[i], F[j]);
        if (dij < d) d = dij;
    }
```

O teste $F[j].y - F[i].y < d$ para o laço interno assim que o vizinho sai do retângulo. Ele é mais econômico que “exatamente 7” e continua limitado por 7. O programa completo está no tutorial da aula.

Para aprofundar: Cormen et al. (2009), §33.4 [5]; Levitin (2012), §5.5 [19].



Exemplo

$A(1, 5)$ $B(2, 9)$ $C(4, 5)$ $D(5, 8)$ |
 $E(6, 8)$ $F(8, 8)$ $G(10, 2)$ $H(11, 9)$

1. Divisa em $x = 5,5$ (entre D e E).
2. Esquerda: $\delta_L = d(A, C) = 3$. Direita: $\delta_R = d(E, F) = 2$. Então $\delta = 2$.
3. Faixa $3,5 < x < 7,5$: $Y' = (C, D, E)$ por y .
4. C : o próximo é D , mas $y_D - y_C = 3 \geq \delta$, e o laço para sem calcular.
5. D : $d(D, E) = 1 < 2$, e o novo δ passa a ser 1.

Resposta: $\{D, E\}$, com $d = 1$, um par que *cruza* a divisa.

Para aprofundar: Exemplo construído para a aula; conferido com o programa do tutorial.

Versão 1: ordena a faixa em cada chamada

$$T(n) = 2T(n/2) + \Theta(n \log n)$$

$f(n) = n \lg n$ não é polinomialmente maior que $n^{\log_2 2} = n$: cai na *lacuna*. Pela extensão do caso 2 ($k = 1$):

$$T(n) = \Theta(n \log^2 n).$$

Versão 2: pré-ordena X e Y uma vez

Separar Y em Y_L , Y_R e filtrar Y' custa $\Theta(n)$:

$$T(n) = 2T(n/2) + \Theta(n)$$

$a = 2 = b^d$: caso 2, $\Theta(n \log n)$.

Somando a pré-ordenação $\Theta(n \log n)$:

$$\Theta(n \log n).$$

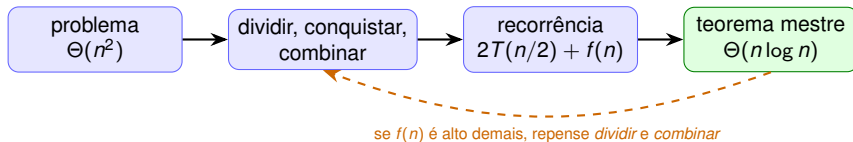
É a mesma recorrência do mergesort. O truque é não pagar a ordenação dentro da recursão, e sim “desintercalar” Y , o inverso do `merge`. Esse limite é ótimo no modelo de árvores de decisão algébricas.

Para aprofundar: Cormen et al. (2009), §33.4 e Exerc. 4.6-2 [5]; Preparata e Shamos (1985), cap. 5 [22].

O que aprendemos

- Dividir pela mediana em x , resolver cada metade e obter $\delta = \min\{\delta_L, \delta_R\}$.
- Pares que cruzam só na faixa $|x - x_m| < \delta$, e cada ponto tem no máximo 7 vizinhos a testar.
- Com a faixa ordenada a cada chamada o custo é $\Theta(n \log^2 n)$; com pré-ordenação, $\Theta(n \log n)$.
- Para $n = 10^6$: de $\approx 5 \times 10^{11}$ distâncias para poucos milhões.

Para aprofundar: Cormen et al. (2009), §33.4 [5]; Kleinberg e Tardos (2006), §5.4 [16].



Próximas aulas: Karatsuba e Strassen, em que o ganho vem de reduzir a , o número de subproblemas, e não $f(n)$.

Para aprofundar: Levitin (2012), §5.4 [19]; Cormen et al. (2009), §4.2 [5].



Alfred V. Aho, John E. Hopcroft, and Jeffrey D. Ullman.

Data Structures and Algorithms.

Addison-Wesley, 1983.

Seção 6.7: o algoritmo das duas buscas para componentes fortemente conexos, atribuído a S. R. Kosaraju (1978, inédito).



Jon Bentley.

Programming Pearls.

Addison-Wesley, 2 edition, 2000.



Manuel Blum, Robert W. Floyd, Vaughan Pratt, Ronald L. Rivest, and Robert E. Tarjan.

Time bounds for selection.

Journal of Computer and System Sciences, 7(4):448–461, 1973.

Artigo original da mediana das medianas: seleção em $O(n)$ no pior caso.



J. Adrian Bondy and U. S. R. Murty.

Graph Theory, volume 244 of *Graduate Texts in Mathematics*.

Springer, 2008.



Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein.

Introduction to Algorithms.

MIT Press, 3 edition, 2009.



Paulo Feofiloff.

Análise de algoritmos.

Material didático online, IME-USP.

Aulas *Divisão e conquista*, *Recorrências* (Teorema Mestre) e *Mergesort*. Acesso em 24 set. 2026.



Paulo Feofiloff.

Algoritmos para grafos (em linguagem C).

Material didático online, DCC/IME-USP, 2020.

Capítulos *Grafos e Estruturas de dados para grafos*. Acesso em 01 set. 2026.



Paulo Feofiloff, Yoshiharu Kohayakawa, and Yoshiko Wakabayashi.

Uma introdução sucinta à teoria dos grafos.

Texto didático online, IME-USP, 2011.

Referência em português para as definições de teoria dos grafos. Acesso em 01 set. 2026.



Michael R. Garey and David S. Johnson.

Computers and Intractability: A Guide to the Theory of NP-Completeness.

W. H. Freeman, 1979.



David Gries.

The Science of Programming.

Springer-Verlag, 1981.



Michael Held and Richard M. Karp.

A dynamic programming approach to sequencing problems.

Journal of the Society for Industrial and Applied Mathematics, 10(1):196–210, 1962.



C. A. R. Hoare.

Algorithm 64: Quicksort.

Communications of the ACM, 4(7):321, 1961.



C. A. R. Hoare.

Algorithm 65: FIND.

Communications of the ACM, 4(7):321–322, 1961.

O quickselect, publicado por Hoare no mesmo ano do quicksort.



C. A. R. Hoare.

An axiomatic basis for computer programming.

Communications of the ACM, 12(10):576–580, 1969.



Arthur B. Kahn.

Topological sorting of large networks.

Communications of the ACM, 5(11):558–562, 1962.

Artigo original do algoritmo de graus de entrada com fila.



Jon Kleinberg and Éva Tardos.

Algorithm Design.

Pearson/Addison-Wesley, 2006.

Capítulo 5, em especial as seções 5.2 (recorrências) e 5.4 (par mais próximo).



Donald E. Knuth.

The Art of Computer Programming, Volume 1: Fundamental Algorithms.

Addison-Wesley, 3 edition, 1997.

Seção 2.2.3: ordenação topológica como aplicação de listas ligadas.



Donald E. Knuth.

The Art of Computer Programming, Volume 3: Sorting and Searching.

Addison-Wesley, 2 edition, 1998.

Seção 5.3.3: seleção mínima em número de comparações.



Anany Levitin.

Introduction to the Design and Analysis of Algorithms.

Pearson, 3 edition, 2012.



Udi Manber.

Introduction to Algorithms: A Creative Approach.

Addison-Wesley, 1989.



David R. Musser.

Introspective sorting and selection algorithms.

Software: Practice and Experience, 27(8):983–993, 1997.

O introselect: híbrido entre quickselect e mediana das medianas.



Franco P. Preparata and Michael Ian Shamos.

Computational Geometry: An Introduction.

Springer, 1985.

Capítulo 5: problemas de proximidade.



Robert Sedgewick.

Algorithms in C, Part 5: Graph Algorithms.

Addison-Wesley, 3 edition, 2002.

Fonte dos exemplos e exercícios do Capítulo 17 citados por Feofiloff.



Robert Sedgewick and Kevin Wayne.

Algorithms.

Addison-Wesley, 4 edition, 2011.



Michael Ian Shamos and Dan Hoey.

Closest-point problems.

In *16th Annual Symposium on Foundations of Computer Science (SFCS 1975)*, pages 151–162. IEEE, 1975.



Micha Sharir.

A strong-connectivity algorithm and its applications in data flow analysis.

Computers & Mathematics with Applications, 7(1):67–72, 1981.

Algoritmo das duas buscas (Kosaraju–Sharir) para componentes fortemente conexos.



Robert Tarjan.

Depth-first search and linear graph algorithms.

SIAM Journal on Computing, 1(2):146–160, 1972.

Artigo que estabeleceu a DFS como ferramenta algorítmica de tempo linear.



Nivio Ziviani.

Projeto de Algoritmos: com implementações em Pascal e C.

Cengage Learning, 3 edition, 2010.



Nivio Ziviani.

Projeto de Algoritmos com Implementações em Pascal e C.

Cengage Learning, 3 edition, 2011.

Capítulo 7: algoritmos em grafos.